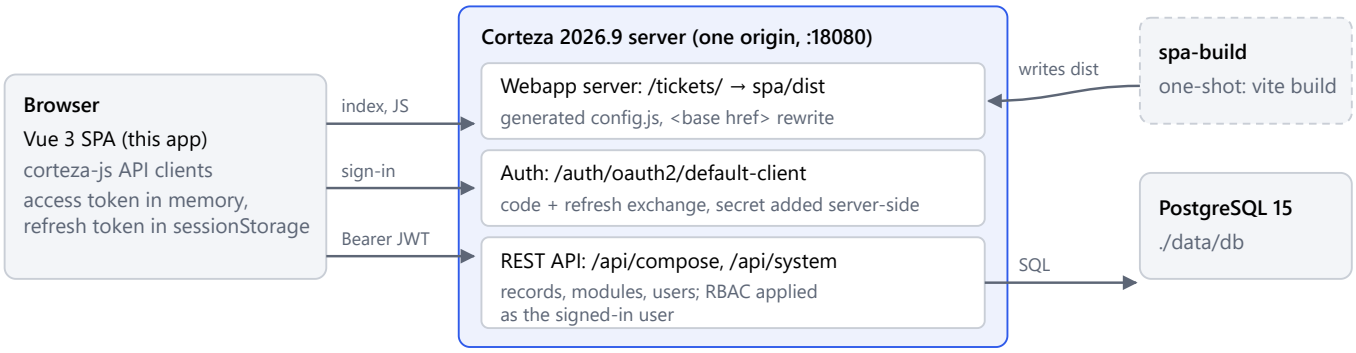
Support desk on Corteza

Vue 3 SPA for Support Tickets and Customers, served by Corteza itself · Martin Marinov · 26 October 2026

CORTEZA	SERVED FROM	FRONT END	PLATFORM TOOLING
2026.9 (Docker, Postgres 15)	`http://localhost:18080/tickets/`	Vue 3.5, Composition API, Vite 6	`@cortezaproject/corteza-js` 2026.9.0

Corteza version. I chose **2026.9.0** (released 19 October 2026). The other maintained line, 2024.9 (last patch August 2026), would leave a new project facing a major upgrade with security improvements early, and the current guide and docs target 2026.9. The cost: a six-day-old .0 release with no patches yet; for production I would pin the exact tag.

1. Architecture



The browser talks to a single origin. Corteza's own **webapp server** serves the built SPA next to its first-party apps (Compose, Admin, Workflow). Sign-in goes through Corteza's **default auth client**, the same flow Corteza's own apps use; every API call carries a JWT issued to the real user, so Corteza's RBAC, the `createdBy` / `updatedBy` / `ownedBy` system fields and the audit trail (Admin → Action log) all refer to that person. A one-shot `spa-build` container builds the app before the server starts, so a fresh clone needs only Docker.

2. Design decisions

2.1 How it is served: Corteza's webapp server

Corteza serves any folder listed in `HTTP_WEBAPP_LIST` from `HTTP_WEBAPP_BASE_DIR` (read in `server/pkg/webapp/serve.go`). For each listed app it generates a `config.js` with the API and auth URLs, rewrites `<base href>` to the mount path, and falls back to `index.html` for client-side routes. The app is built with relative asset paths and reads its URLs from `config.js` at runtime, so **one build works on any host or path**.

Option	Assessment
Corteza webapp server (chosen)	Same origin as API and auth: no CORS, no second server, and the only option where the built-in default auth client works without extra configuration.
Separate static host + CORS	Works via `HTTP_CORS_ALLOWED_ORIGINS`, but adds a second deployment, CORS to maintain, and a whitelisted redirect URI (`AUTH_DEFAULT_REDIRECT_URIS`) or a dedicated auth client.
Reverse proxy (nginx)	Re-creates what Corteza already does (same origin, SPA fallback) as another moving part.
Embed in a Compose page	Not tried. An iframe inside Compose fights the app's own layout and keyboard focus; it suits widgets more than a full app.
App launcher entry	Corteza can list applications in its launcher. That complements hosting rather than replacing it; not set up here.

Delivery. Locally `spa/dist` is bind-mounted, but Corteza reads `index.html` **once at start-up**: after a rebuild it serves the old one (pointing at deleted assets) until restarted, so `npm run deploy` builds and restarts together. For production I would bake the build into a derived image (multi-stage Dockerfile `FROM cortezaproject/corteza:2026.9.0`, `dist`

copied to `/corteza/webapp/tickets`): one versioned artifact, a fresh container per deploy, no stale `index.html`. `HTTP_WEBAPP_LIST` replaces Corteza's default list, so it needs checking on each upgrade.

2.2 Acting as the signed-in user: the default auth client

`/auth/oauth2/default-client` starts the authorization-code flow and, on POST, exchanges the code or refresh token, **adding the default client's secret on the server**; it only accepts redirect URIs on Corteza's own origin (plus `AUTH_DEFAULT_REDIRECT_URIS`). No client ID or secret lives in the app. The access token stays in memory and is refreshed at 75% of its lifetime; the refresh token is kept in `sessionStorage`, so a reload resumes the session and closing the tab ends it. **Checked:** a ticket created and deleted through the app carries the user as `createdBy` / `ownedBy` / `deletedBy` and appears in Admin → Action log under that user (needs `ACTIONLOG_ENABLED=true`, 4.4).

Why not a dedicated auth client: Corteza gives every client a secret and checks it at the token endpoint (there is no public-client mode), so a browser app would have to ship that secret or add a token-exchange backend; the default-client endpoint is that backend. The cost: Corteza sees this app as its own web-app client (`corteza-webapp`), so it cannot be disabled, role-limited or time-limited apart from Compose and Admin. If it needed its own policy, I would put a dedicated client behind a small token-exchange endpoint.

2.3 Using the platform's client tooling, and where I did not

Corteza provides	Decision
`corteza-js`: API clients, `compose.Module` / `Record`, `RecordValidator`	**Used.** All API calls go through `apiClients.Compose` / `System`; records are `compose.Record` objects; required-field checks use `RecordValidator`. Cost: the library is not tree-shakeable, about 178 kB gzipped.
`corteza-vue`: auth plugin and UI components	**Not used, because it requires Vue 2.7 and bootstrap-vue** and cannot run in a Vue 3 app. Its sign-in flow was ported (~150 lines, same endpoint and storage model); UI components are new.
Compose's own record list and editor, which work for any module	**Re-implemented in part, deliberately.** The brief asks for a separate SPA, so list and form are rebuilt, but generated from the module definition (fields, options, defaults, Bool labels, option colours, permissions) as Compose's are: a new field needs no code change, and Customer is found through the ticket's Record field. Cost: a generic, not ticket-specific UI.
System fields `createdBy/At`, `updatedBy/At`, `ownedBy`	**Shown, not re-authored**: list columns and the dialog header, with user IDs resolved to names in one `userList` call.

2.4 UI and accessibility patterns

- **Hand-rolled styling, no component library.** The UI needs a table, cards, dialogs and one combobox; a library would add more weight than corteza-js already does. Colours are tokens checked with a script: text ≥ 4.5:1 and control borders ≥ 3:1 (WCAG 1.4.11), in light and dark themes.
- **Modals on the native `<dialog>`** (`showModal()`): the browser provides the focus trap, an inert background and Esc. The component adds initial focus, focus return (to the page heading if the opener row was deleted), a busy state that blocks closing mid-save, and full-screen layout on phones.
- **Forms:** errors are linked with `aria-invalid` / `aria-describedby`, and focus moves to the first invalid field. Server-side validation errors (HTTP 200 with `error.details[].meta.field`) map to the same fields.
- **Customer picker:** WAI-ARIA combobox with a listbox popup and `aria-activedescendant`; results announced politely; only records chosen from the list can be stored, because Corteza itself accepts any ID in a Record field.
- **One layout at a time:** a table above 720 px and cards below, chosen by `matchMedia` rather than CSS hiding, so screen readers and Tab order never see two copies. Sortable headers carry `aria-sort`; the card view has a labelled "Sort by" control.
- **List state in the address:** search, sort and page come only from the URL, so links, reloads and Back/Forward agree; typing replaces the history entry.
- **Permission-aware:** "New", Edit/View/Delete, hidden and read-only fields follow the flags Corteza returns with each module, field and record.

3. User guide

Open **http://localhost:18080/tickets/** and choose **Sign in with Corteza**. If you are already signed in to Corteza you go straight in. **Sign out** (top right) ends the app session and sends you to Corteza's sign-out.

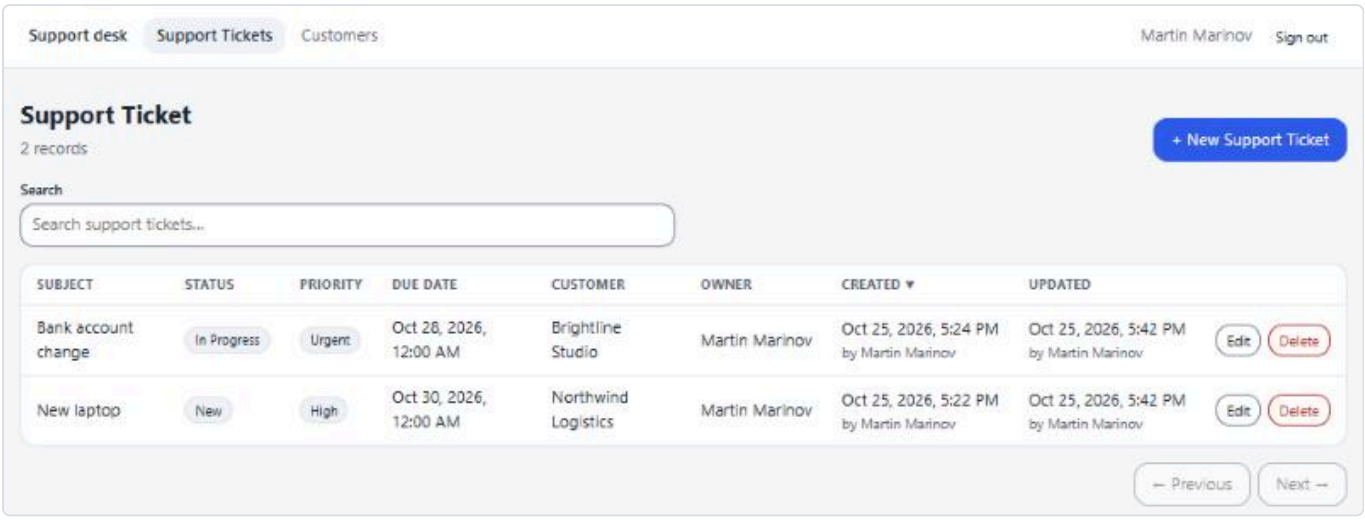


Figure 1. Support tickets. The top bar switches between Support Tickets and Customers.

Find tickets

- **Search** matches text fields (subject, description) as you type.
- **Sort** by clicking a column header; click again to reverse. On a phone use **Sort by**.
- **Previous / Next** page through 20 tickets at a time. Search, sort and page are kept in the address, so a link reopens the same view and Back returns to the previous one.
- **Owner, Created, Updated** come from Corteza and show who did what.

Create or edit a ticket

- **+ New Support Ticket** opens a form; **Edit** on a row opens the same form filled in. Fields marked * are required.
- **Customer:** type part of a name, pick with the mouse or ↓ ↑ and Enter. Only existing customers can be chosen; ✕ clears it.
- **Create** / **Save changes** stores the ticket under your name. Problems are listed and the cursor moves to the first field to fix.
- If you may only read a ticket, the button says **View** and the form is read-only.

Delete

Delete asks for confirmation; **Cancel** is the default. Deleting a customer warns how many tickets still refer to it (or that it could not check); those tickets then show “Missing record”.

Customers

The **Customers** section works the same way: list, search, create, edit, delete.

Keyboard and phone

Everything works by keyboard: Tab between controls, Enter to activate, Esc to close a list or dialog (unsaved input in a closed form is discarded). On narrow screens tickets show as cards and forms fill the screen.

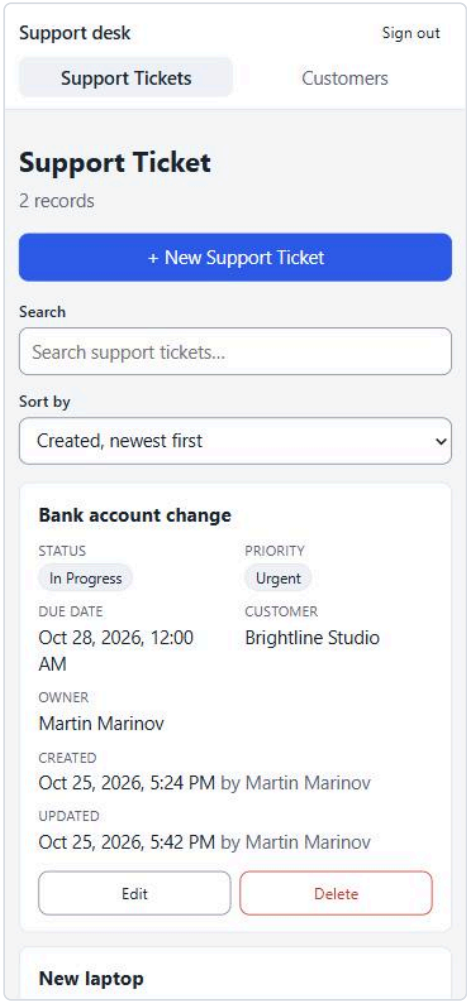


Figure 2. The same list at phone width.

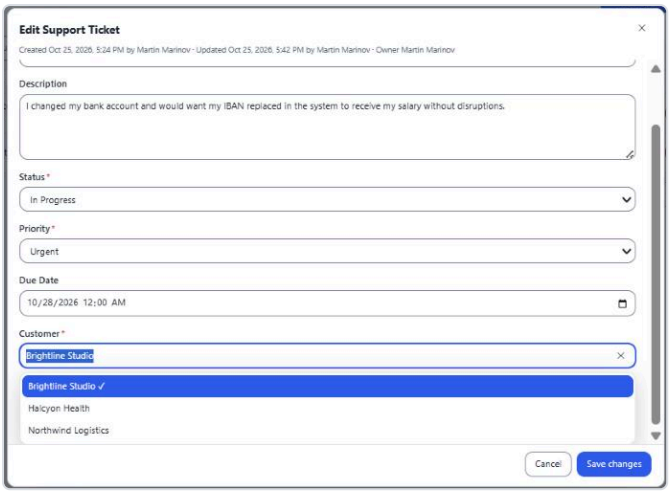


Figure 3. Editing a ticket; the customer picker is open.

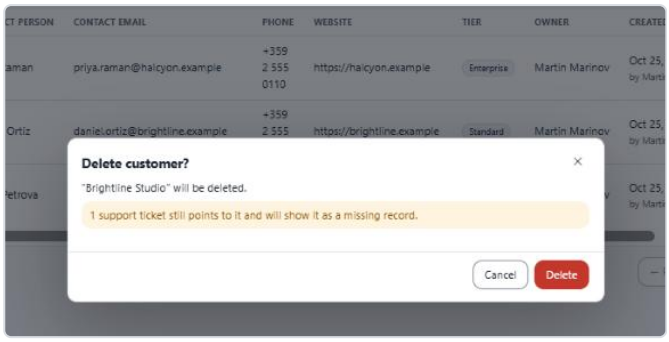


Figure 4. Deleting a customer that a ticket still uses.

4. Admin guide

4.1 Install and run

1. `cp .env.example .env` and set `AUTH_JWT_SECRET` (e.g. `openssl rand -hex 32`).
2. `docker compose up -d`. This builds the SPA (`spa-build`), then starts Postgres and Corteza on `http://localhost:18080`.
3. Open Corteza and sign up; the first account becomes super-admin. No credentials are shipped.
4. Configure the namespace and modules (4.2), then open `/tickets/`.

4.2 Namespace and modules (Low Code builder, no code)

Namespace slug `plexys-hw`. Create **Customer** first, because the ticket's Record field needs it. Required fields are marked *.

Module	Fields
Customer `customer`	`name` * String · `contact_name` String · `contact_email` * Email · `phone` String · `website` Url · `tier` * Select (standard, premium, enterprise; default standard) · `active` Bool (default on) · `notes` String, multi-line
Support Ticket `ticket`	`subject` * String · `description` String, **multi-line** (not "multiple values") · `status` * Select (New, In Progress, Resolved, Closed) · `priority` * Select (Low, Medium, High, Urgent) · `due_date` Date/Time · `customer` * Record → Customer, label field `name`

4.3 Authentication and permissions

- No auth client to create: the app uses Corteza's default client. Its redirect URI is on Corteza's own origin, which Corteza always allows.
- For the dev server on `:5173`, Corteza must allow that origin: `AUTH_DEFAULT_REDIRECT_URIS=http://localhost:5173` (in `.env.example`).
- Access is controlled by Corteza roles and the permissions set on the namespace and modules. The app hides "New" without create permission, offers View instead of Edit without update permission, hides Delete without delete permission, and hides or locks individual fields per field permission.

4.4 Configuration (root `.env`; host builds read `spa/.env`)

Variable	Purpose
`VERSION` = `2026.9`	Corteza image tag; follows the 2026.9 patch releases (pin e.g. `2026.9.0` for production). Keep `@cortezaproject/corteza-js` on the same version.
`DOMAIN`, `HTTP_WEBAPP_ENABLED`	Host and port Corteza is reached at (`localhost:18080`), and switching on its webapp server, which serves this app.

`ACTIONLOG_ENABLED = true`	Corteza's audit trail (Admin → Action log): who created, changed or deleted each record. Corteza's default; set explicitly so it is not turned off.
`HTTP_WEBAPP_LIST`	`admin,compose,workflow,reporter,privacy,tickets`. `tickets` is this app's folder and URL path.
`VITE_NAMESPACE`	Namespace slug. Build-time: rebuild after changing. The `spa-build` container takes it from the root `.env`; `npm run deploy` and `npm run dev` take it from `spa/.env`.
`AUTH_DEFAULT_REDIRECT_URIS`	Extra allowed sign-in origins; only needed for the dev server.
`AUTH_JWT_SECRET`, `DB_*`	Corteza token signing secret and Postgres credentials (local defaults).

4.5 Operating it

- **Deploy a change:** `cd spa && npm run deploy`; once beforehand, `cp .env.example .env` in `spa/`, or the build has no namespace slug. Without the restart Corteza keeps serving the previous `index.html` and the page stays blank.
- **Upgrade Corteza:** change `VERSION`, bump `corteza-js` to match, rebuild, `docker compose up -d`.
- **Data:** everything is in `./data/` (Postgres and Corteza files); back up that folder with the stack stopped.
- **Troubleshooting:** "Namespace … not found" → slug or the user's access to the namespace; sent back to sign-in → refresh token rejected (also happens while Postgres is unavailable); blank page after a rebuild → restart the server.

5. Known limitations and next steps

Limitation	Next step
Restricted-user paths (View-only dialog, hidden/locked fields, no "New") were only exercised as super-admin, where every permission flag is true.	Create a role with read-only access and walk through the app as a second user.
Server-side validation errors are mapped to fields per the source code, but never triggered from the UI (client checks catch required fields first).	Add a field with a server rule (e.g. unique value) and test.
Deleting a customer still used by tickets is warned about, not prevented; Corteza has no referential integrity for Record fields.	Block it server-side with a workflow, or offer to reassign the tickets.
Status and priority sort by stored value (high, low, medium, urgent), not by severity.	Add a numeric sort field or sort expression in the module.
A customer's tickets are not listed on the customer; the Ticket Comment module exists in the instance but has no UI.	"Tickets" count with a filtered link; comments inside the ticket dialog.
Status, priority and tier badges all look the same: no Select option has a colour.	Set option colours in the builder (e.g. New blue, In Progress amber, Resolved green, Urgent red), or a value → colour fallback with light/dark variants in the app.
The list has free-text search only; it cannot be filtered by a field such as status, priority, due date or customer.	Filter controls generated per field kind (choices for Select/Bool, a range for DateTime, the Record picker), combined into the Corteza query and kept in the address.
Closing a form with unsaved changes discards them without asking.	Ask before discarding a dirty form.
corteza-js adds ~178 kB gzipped (no tree-shaking). Setup needs the modules created by hand.	Lazy-load the client; ship a namespace export for one-step import.